

Numerical Recipes Software 2007, "Implementation of the Euler Transformation," *Numerical Recipes Webnote No. 5*, at <http://www.nr.com/webnotes?5> [7]

Jentschura, U.D., Mohr, P.J., Soff, G., and Weniger, E.J. 1999, "Convergence Acceleration via Combined Nonlinear-Condensation Transformations," *Computer Physics Communications*, vol. 116, pp. 28–54.[8]

Dahlquist, G., and Björck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), Chapter 3.

5.4 Recurrence Relations and Clenshaw's Recurrence Formula

Many useful functions satisfy recurrence relations, e.g.,

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x) \quad (5.4.1)$$

$$J_{n+1}(x) = \frac{2n}{x}J_n(x) - J_{n-1}(x) \quad (5.4.2)$$

$$nE_{n+1}(x) = e^{-x} - xE_n(x) \quad (5.4.3)$$

$$\cos n\theta = 2\cos\theta\cos(n-1)\theta - \cos(n-2)\theta \quad (5.4.4)$$

$$\sin n\theta = 2\cos\theta\sin(n-1)\theta - \sin(n-2)\theta \quad (5.4.5)$$

where the first three functions are Legendre polynomials, Bessel functions of the first kind, and exponential integrals, respectively. (For notation see [1].) These relations are useful for extending computational methods from two successive values of n to other values, either larger or smaller.

Equations (5.4.4) and (5.4.5) motivate us to say a few words about trigonometric functions. If your program's running time is dominated by evaluating trigonometric functions, you are probably doing something wrong. Trig functions whose arguments form a linear sequence $\theta = \theta_0 + n\delta$, $n = 0, 1, 2, \dots$, are efficiently calculated by the recurrence

$$\begin{aligned} \cos(\theta + \delta) &= \cos\theta - [\alpha\cos\theta + \beta\sin\theta] \\ \sin(\theta + \delta) &= \sin\theta - [\alpha\sin\theta - \beta\cos\theta] \end{aligned} \quad (5.4.6)$$

where α and β are the precomputed coefficients

$$\alpha \equiv 2\sin^2\left(\frac{\delta}{2}\right) \quad \beta \equiv \sin\delta \quad (5.4.7)$$

The reason for doing things this way, rather than with the standard (and equivalent) identities for sums of angles, is that here α and β do not lose significance if the incremental δ is small. Likewise, the adds in equation (5.4.6) should be done in the order indicated by the square brackets. We will use (5.4.6) repeatedly in Chapter 12, when we deal with Fourier transforms.

Another trick, occasionally useful, is to note that both $\sin\theta$ and $\cos\theta$ can be calculated via a single call to $\tan$:

$$t \equiv \tan\left(\frac{\theta}{2}\right) \quad \cos\theta = \frac{1-t^2}{1+t^2} \quad \sin\theta = \frac{2t}{1+t^2} \quad (5.4.8)$$

The cost of getting both $\sin$ and $\cos$, if you need them, is thus the cost of $\tan$ plus 2 multiplies, 2 divides, and 2 adds. On machines with slow trig functions, this can be a savings. *However*, note that special treatment is required if $\theta \rightarrow \pm\pi$. And also note that many modern machines have *very fast* trig functions; so you should not assume that equation (5.4.8) is faster without testing.

5.4.1 Stability of Recurrences

You need to be aware that recurrence relations are not necessarily *stable* against roundoff error in the direction that you propose to go (either increasing n or decreasing n). A three-term linear recurrence relation

$$y_{n+1} + a_n y_n + b_n y_{n-1} = 0, \quad n = 1, 2, \dots \quad (5.4.9)$$

has two linearly independent solutions, f_n and g_n , say. Only one of these corresponds to the sequence of functions f_n that you are trying to generate. The other one, g_n , *may* be exponentially growing in the direction that you want to go, or exponentially damped, or exponentially neutral (growing or dying as some power law, for example). If it is exponentially growing, then the recurrence relation is of little or no practical use in that direction. This is the case, e.g., for (5.4.2) in the direction of increasing n , when $x < n$. You cannot generate Bessel functions of high n by forward recurrence on (5.4.2).

To state things a bit more formally, if

$$f_n/g_n \rightarrow 0 \quad \text{as} \quad n \rightarrow \infty \quad (5.4.10)$$

then f_n is called the *minimal* solution of the recurrence relation (5.4.9). Nonminimal solutions like g_n are called *dominant* solutions. The minimal solution is unique, if it exists, but dominant solutions are not — you can add an arbitrary multiple of f_n to a given g_n . You can evaluate any dominant solution by forward recurrence, *but not the minimal solution*. (Unfortunately it is sometimes the one you want.)

Abramowitz and Stegun (in their Introduction!)[1] give a list of recurrences that are stable in the increasing or decreasing direction. That list does not contain all possible formulas, of course. Given a recurrence relation for some function $f_n(x)$, you can test it yourself with about five minutes of (human) labor: For a fixed x in your range of interest, start the recurrence not with true values of $f_j(x)$ and $f_{j+1}(x)$, but (first) with the values 1 and 0, respectively, and then (second) with 0 and 1, respectively. Generate 10 or 20 terms of the recursive sequences in the direction that you want to go (increasing or decreasing from j), for each of the two starting conditions. Look at the differences between the corresponding members of the two sequences. If the differences stay of order unity (absolute value less than 10, say), then the recurrence is stable. If they increase slowly, then the recurrence may be mildly unstable but quite tolerably so. If they increase catastrophically, then there is an exponentially growing solution of the recurrence. If you know that the function that you want actually corresponds to the growing solution, then you can keep the recurrence formula anyway (e.g., the case of the Bessel function $Y_n(x)$ for increasing n ; see §6.5). If you don't know which solution your function corresponds to, you must at this point reject the recurrence formula. Notice that you can do this test *before* you go to the trouble of finding a numerical method for computing the two